

ZeroWasteML

MANUAL DE REFERENCIA DE UN LENGUAJE DE MARCAS PARA LA GESTIÓN DE ALIMENTOS

Autor: Mary Joy Manalansan
Módulo: Lenguaje de Marcas
Ciclo: ASIR1D
Curso: 2026-2027

Contenido

1. Presentación del lenguaje 2
1.1. ¿Qué es ZeroWasteML? 2
1.2. ¿Qué lo diferencia? 2
2. Ámbitos de aplicación 3
2.1. Uso doméstico 3
2.2. Restaurantes y pequeños negocios 4
2.3. Almacenamiento e intercambio de información 4
2.4. Supuesto práctico: donación a un banco de alimentos 4
3. Diseño y características 5
3.1. Extensión de los archivos 5
3.2. Elemento raíz 6
3.3. Elemento <despensa> 6
3.4. Elemento <alimento> 7
3.5. Elemento <nombre> 7
3.6. Elemento <ubicacion> 8
3.7. Elemento <caducidad> 8
3.8. Elemento <apertura> 9
3.9. Elemento <notas> 9
3.10. Reglas de sintaxis 9
3.11. Tipos de alimentos 10
3.12. Reglas de validación 11
4. Ejemplos de documentos 12
4.1. Ejemplo 1: Documento sencillo 12
4.2. Ejemplo 2: Documento intermedio 12
4.3. Ejemplo 3: Documento completo 13
5. Ventajas y limitaciones 15
5.1. Ventajas 15
5.2. Limitaciones 15
5.3. Resumen de utilidad 16
6. Inspiración y referencias 17
Conclusión 18

1. Presentación del lenguaje

1.1. ¿Qué es ZeroWasteML?

ZeroWasteML (Zero Waste Markup Language) es un lenguaje de marcas diseñado para organizar y almacenar información sobre los alimentos que tenemos en casa.

Su objetivo es facilitar la gestión de alimentos de la nevera, el congelador o la despensa, almacenando información como:

- Nombre del alimento
- Tipo de alimento
- Cantidad
- Ubicación
- Fecha de caducidad
- Fecha de apertura
- Estado del alimento
- Notas adicionales

ZeroWasteML es un lenguaje basado en XML, pero utiliza elementos y atributos específicos para representar información relacionada con los alimentos.

Por ejemplo:

```
<alimento tipo="lacteo" cantidad="3" unidad="ud">
  <nombre>Yogur natural</nombre>
  <ubicacion>nevera</ubicacion>
  <caducidad>2026-09-28</caducidad>
</alimento>
```

La idea surgió de un problema real en mi casa: muchas veces se nos caducan productos en la nevera sin darnos cuenta, porque nadie recuerda qué se compró ni cuándo. Por eso quise diseñarlo de forma que la fecha de caducidad fuera lo más importante.

1.2. ¿Qué lo diferencia?

ZeroWasteML utiliza XML como base, igual que otros lenguajes como SVG, para gráficos, RSS, para noticias, o XHTML, para páginas web. La idea es aprovechar XML, pero adaptándolo a algo más concreto: organizar los alimentos que tenemos en casa.

Lo que aporta ZeroWasteML es un vocabulario propio (etiquetas, atributos y valores permitidos) y unas reglas de validación pensadas solo para alimentos.

Frente a otros lenguajes de marcas existentes:

- XML genérico permite cualquier etiqueta, pero no dice cuáles usar para describir un alimento. ZeroWasteML fija las etiquetas y los valores, de modo que dos aplicaciones distintas guardan la información de la misma forma.
- Cooklang es un lenguaje de marcas real para escribir recetas. Me sirvió de inspiración, pero está pensado para describir cómo se cocina, no para controlar qué alimentos hay en casa.
- HTML sirve para mostrar información en un navegador, no para almacenar datos estructurados.

Lo que más lo diferencia es que distingue el tipo de fecha. En las etiquetas de los alimentos no es lo mismo la fecha de caducidad (pasada esa fecha el alimento puede no ser seguro) que la fecha de consumo preferente (pasada esa fecha sigue siendo seguro, aunque puede perder sabor o textura). Mucha comida se tira por confundir las dos, así que ZeroWasteML las distingue con el atributo tipo del elemento <caducidad>.

Gracias a esta estructura, una aplicación externa puede leer un archivo .zwml y, por ejemplo:

- Ordenar los alimentos según su fecha de caducidad.
- Mostrar los alimentos que caducan próximamente.
- Buscar alimentos por ubicación o tipo.

2. Ámbitos de aplicación

ZeroWasteML puede utilizarse principalmente en situaciones donde sea necesario organizar información sobre alimentos.

2.1. Uso doméstico

Una familia podría utilizar un archivo ZeroWasteML para guardar los alimentos que tiene en casa.

Por ejemplo:

```
<despensa nombre="Casa">
  <alimento tipo="lacteo" cantidad="4" unidad="ud">
    <nombre>Yogur natural</nombre>
    <ubicacion>nevera</ubicacion>
    <caducidad>2026-09-28</caducidad>
  </alimento>
```

</despensa>

Una aplicación podría leer este archivo y mostrar los productos que están próximos a caducar.

2.2. Restaurantes y pequeños negocios

También podría utilizarse en restaurantes, cafeterías o pequeños comercios para organizar productos almacenados.

Por ejemplo, se podrían registrar:

- Alimentos almacenados.
- Cantidades disponibles.
- Fechas de caducidad.
- Ubicación de los productos.

Esto permitiría que una aplicación externa consultase los datos y ayudase a controlar los productos que deben utilizarse antes.

2.3. Almacenamiento e intercambio de información

Un archivo.zwml podría utilizarse para guardar y compartir información entre diferentes aplicaciones.

Por ejemplo:

Aplicación A → despensa.zwml →Aplicación B

Al utilizar una estructura definida, diferentes programas podrían interpretar los mismos datos.

2.4. Supuesto práctico: donación a un banco de alimentos

Desde 2025 está en vigor en España la Ley 1/2025 de prevención de las pérdidas y el desperdicio alimentario, que establece que los alimentos no vendidos que siguen siendo aptos para el consumo deben donarse antes que tirarse. Para ello, supermercados y bancos de alimentos necesitan intercambiar información sobre productos a punto de caducar.

Supuesto: un supermercado de barrio de Madrid usa un programa de gestión de almacén. Cada lunes, ese programa genera automáticamente un archivo donacion.zwml con los productos que caducan esa semana y con los que han pasado su fecha de consumo preferente pero siguen siendo aptos. El archivo se envía al banco de alimentos, que usa un programa de otra empresa. Como los dos programas entienden ZeroWasteML, el banco de alimentos importa el archivo sin teclear nada a mano y puede:

- Separar los productos con fecha de caducidad (hay que repartirlos en pocos días) de los de consumo preferente (hay más margen).
- Saber si necesita transporte refrigerado mirando la ubicación (nevera o congelador).
- Planificar el reparto según las cantidades.

Sin un formato común, cada supermercado enviaría la lista a su manera (un Excel, un PDF, un correo) y alguien tendría que pasarla a mano.

Ejemplo del archivo que enviaría el supermercado:

```
<despensa nombre="Supermercado Barrio Delicias" fecha="2026-09-28">
  <!-- Caducan esta semana: repartir primero -->
  <alimento tipo="lacteo" cantidad="24" unidad="ud" estado="cerrado">
    <nombre>Yogur natural</nombre>
    <ubicacion>nevera</ubicacion>
    <caducidad>2026-10-01</caducidad>
  </alimento>
  <!-- Consumo preferente pasado: sigue siendo apto -->
  <alimento tipo="cereal" cantidad="15" unidad="kg" estado="cerrado">
    <nombre>Arroz redondo</nombre>
    <ubicacion>despensa</ubicacion>
    <caducidad tipo="preferente">2026-09-20</caducidad>
  </alimento>
</despensa>
```

3. Diseño y características

3.1. Extensión de los archivos

Los archivos creados con ZeroWasteML utilizarán la extensión: **.zwml**

Por ejemplo:

despensa.zwml

3.2. Elemento raíz

Todo documento ZeroWasteML debe tener un único elemento raíz llamado:

<despensa>

Dentro de este elemento se pueden incluir uno o varios elementos <alimento>.

Ejemplo:

```
<despensa>
  <alimento>
    ...
  </alimento>
</despensa>
```

El elemento raíz se llama <despensa> porque representa el conjunto de alimentos almacenados en un lugar (una casa, un restaurante o un supermercado), no el mueble. El mueble concreto se indica con <ubicacion>despensa</ubicacion>.

La jerarquía de elementos es la siguiente (entre paréntesis, cuántas veces puede aparecer cada uno):

```
despensa (1)
└── alimento (1 o más)
        nombre (1, obligatorio)
        ubicacion (1, obligatorio)
        caducidad (1, obligatorio)
        apertura (0 o 1, opcional)
        notas (0 o 1, opcional)
```

3.3. Elemento <despensa>

Es el elemento raíz del documento.

Puede tener los siguientes atributos:

Atributo	Obligatorio	Descripción
nombre	No	Nombre de la despensa o ubicación general
fecha	No	Fecha en la que se actualizó el documento

Ejemplo:

<despensa nombre="Casa" fecha="2026-09-23">

3.4. Elemento <alimento>

Representa un alimento almacenado.

Tiene los siguientes atributos:

Atributo	Obligatorio	Descripción
tipo	Sí	Tipo de alimento
cantidad	Sí	Cantidad disponible
unidad	Sí	Unidad utilizada
estado	No	Estado del alimento

Ejemplo:

<alimento tipo="lacteo" cantidad="3" unidad="ud" estado="cerrado">

Los valores permitidos para estado pueden ser:

- cerrado
- abierto
- cocinado

Los valores permitidos para unidad son:

- ud (unidades)
- g (gramos)
- kg (kilogramos)
- ml (mililitros)
- l (litros)

3.5. Elemento <nombre>

Indica el nombre del alimento.

Es obligatorio dentro de <alimento>.

Ejemplo: <nombre>Pechuga de pollo entera</nombre>

3.6. Elemento <ubicacion>

Indica dónde se encuentra almacenado el alimento.

Los valores que se pueden utilizar son:

- nevera
- congelador
- despensa
- frutero

Ejemplo:

```
<ubicacion>nevera</ubicacion>
```

3.7. Elemento <caducidad>

Indica la fecha de caducidad del alimento.

ZeroWasteML utiliza la **representación de fecha calendario de ISO 8601**, concretamente el formato: **AAAA-MM-DD**

Donde:

- AAAA = año
- MM = mes
- DD = día

Por ejemplo:

```
<caducidad>2026-09-25</caducidad>
```

Esta fecha corresponde al **25 de septiembre de 2026**.

Se utiliza este formato porque es claro, internacional y evita confusiones entre diferentes formas de escribir una fecha. Además, al utilizar año-mes-día, las fechas pueden ordenarse fácilmente de forma cronológica.

Atributo tipo

El elemento <caducidad> admite un atributo opcional, tipo, que indica qué clase de fecha es:

- caducidad (valor por defecto): fecha límite de seguridad. Pasada esta fecha, el alimento no se debe consumir. La llevan, por ejemplo, la carne, el pescado fresco o la leche pasteurizada.

- preferente: fecha de consumo preferente. Pasada esta fecha el alimento sigue siendo seguro, pero puede perder calidad. La llevan, por ejemplo, el arroz, la pasta o las conservas.
- estimada: fecha que pone el propio usuario para alimentos que no traen fecha en la etiqueta, como la fruta y la verdura fresca.

Si no se escribe el atributo, se entiende que es tipo="caducidad".

Ejemplo:

<caducidad tipo="preferente">2027-05-10</caducidad>

3.8. Elemento <apertura>

- Indica la fecha en la que se abrió un alimento
- También utiliza el formato ISO 8601 AAAA-MM-DD

Ejemplo:

<apertura>2026-09-20</apertura>

Este elemento es opcional porque no todos los alimentos necesitan registrar una fecha de apertura.

Solo puede aparecer si el alimento tiene estado="abierto".

3.9. Elemento <notas>

Permite añadir información adicional sobre el alimento.

Ejemplo:

<notas> Una vez abierto, consumir en 3 días.</notas>

Es un elemento opcional.

3.10. Reglas de sintaxis

ZeroWasteML utiliza varias reglas inspiradas en XML.

Regla 1: Un único elemento raíz

Todo documento debe comenzar y terminar con <despensa>.

Regla 2: Las etiquetas deben cerrarse

Ejemplo correcto:

<nombre>Manzana</nombre>

Incorrecto: <nombre>Manzana

Regla 3: Las etiquetas deben estar correctamente anidadas

Correcto:

```
<alimento>
    <nombre>Manzana</nombre>
    <ubicacion>frutero</ubicacion>
</alimento>
```

Incorrecto:

```
<alimento>
    <nombre>Manzana</alimento>
</nombre>
```

Regla 4: Las etiquetas se escriben en minúsculas

Por ejemplo:

<caducidad>

Regla 5: Los atributos deben escribirse entre comillas

Por ejemplo:

<alimento tipo="fruta" cantidad="5" unidad="ud">

Regla 6: Se pueden utilizar comentarios

Los comentarios utilizan la misma sintaxis que XML:

<!-- Alimentos de la nevera -->

Los comentarios no forman parte de los datos procesados.

Regla 7: Sin tildes ni "ñ" en etiquetas ni en valores predefinidos

Los nombres de etiquetas y atributos, y los valores predefinidos, se escriben sin tildes ni ñ para evitar problemas de codificación entre programas. Por ejemplo, se escribe <ubicacion> y tipo="lacteo". El texto libre, como el contenido de <nombre> o <notas>, sí puede llevar tildes.

3.11. Tipos de alimentos

El atributo tipo puede utilizar diferentes valores predefinidos:

- fruta
- verdura

- lacteo
- carne
- pescado
- conserva
- cereal
- bebida
- otro

Ejemplo:

<alimento tipo="fruta" cantidad="4" unidad="ud">

3.12. Reglas de validación

ZeroWasteML distingue dos niveles de corrección:

- **Documento bien formado:** cumple las reglas de sintaxis del apartado 3.10 (un único elemento raíz, etiquetas cerradas y bien anidadas, minúsculas, atributos entre comillas y sin tildes). Cualquier analizador XML puede comprobarlo.
- **Documento válido:** además de estar bien formado, cumple estas reglas propias de ZeroWasteML:

1. El elemento raíz es <despensa> y contiene al menos un <alimento>.
2. Cada <alimento> tiene los atributos tipo, cantidad y unidad.
3. Los atributos tipo, unidad y estado solo usan los valores permitidos (apartados 3.4 y 3.11).
4. La cantidad es un número mayor que cero.
5. Los hijos de <alimento> aparecen en el orden indicado en el apartado 3.2, y <nombre>, <ubicacion> y <caducidad> son obligatorios.
6. <ubicacion> solo admite nevera, congelador, despensa o frutero.
7. Todas las fechas usan el formato AAAA-MM-DD.
8. El atributo tipo de <caducidad> solo admite caducidad, preferente o estimada.
9. <apertura> solo puede aparecer si el alimento tiene estado="abierto".
10. Si la fecha es de tipo caducidad, la fecha de <apertura> no puede ser posterior a ella.

4. Ejemplos de documentos

4.1. Ejemplo 1: Documento sencillo

Este ejemplo muestra la estructura básica de ZeroWasteML utilizando un único alimento.

```
<despensa>
	<!-- Ejemplo 1: estructura mínima de un documento ZeroWasteML -->
	<!-- Un solo alimento con sus atributos y elementos obligatorios -->
	<alimento tipo="lacteo" cantidad="1" unidad="ud">
		<nombre>Leche entera</nombre>
		<ubicacion>nevera</ubicacion>
		<caducidad>2026-09-28</caducidad>
	</alimento>
</despensa>
```

En este ejemplo se muestran:

- Elemento raíz
- Elemento alimento
- Atributos
- Nombre
- Ubicación
- Fecha de caducidad

4.2. Ejemplo 2: Documento intermedio

Este ejemplo incluye varios alimentos y diferentes tipos de productos.

```
	<despensa nombre="Casa">
		<!-- Ejemplo 2: varios alimentos, tipos, unidades y ubicaciones -->
		<alimento tipo="carne" cantidad="500" unidad="g">
			<nombre>Pechuga de pollo</nombre>
			<ubicacion>nevera</ubicacion>
			<caducidad>2026-09-27</caducidad>
		</alimento>
```

```
        <!-- Uso del atributo opcional estado -->
        <alimento tipo="lacteo" cantidad="3" unidad="ud" estado="cerrado">
            <nombre>Yogur natural</nombre>
            <ubicacion>nevera</ubicacion>
            <caducidad>2026-09-28</caducidad>
        </alimento>
        <!-- El arroz lleva fecha de consumo preferente, no de caducidad -->
        <alimento tipo="cereal" cantidad="1" unidad="kg">
            <nombre>Arroz</nombre>
            <ubicacion>despensa</ubicacion>
            <caducidad tipo="preferente">2027-05-10</caducidad>
        </alimento>
    </despensa>
```

Este ejemplo muestra:

- Varios alimentos
- Diferentes tipos
- Diferentes ubicaciones
- Cantidades
- Estados y fecha de caducidad

4.3. Ejemplo 3: Documento completo

Este ejemplo utiliza prácticamente todos los elementos definidos por ZeroWasteML.

```
<despensa nombre="Casa" fecha="2026-09-23">
  <!-- Productos de la nevera -->
  <alimento tipo="lacteo" cantidad="1" unidad="ud" estado="abierto">
    <nombre>Leche entera</nombre>
    <ubicacion>nevera</ubicacion>
    <caducidad>2026-09-27</caducidad>
    <apertura>2026-09-22</apertura>
    <notas>Conservar refrigerada. </notas>
  </alimento>

  <alimento tipo="verdura" cantidad="5" unidad="ud">
    <nombre>Zanahoria</nombre>
    <ubicacion>nevera</ubicacion>
    <caducidad tipo="estimada">2026-10-02</caducidad>
```

```
    </alimento>
    <!-- Productos del congelador -->
    <alimento tipo="carne" cantidad="500" unidad="g">
        <nombre>Pechuga de pollo</nombre>
        <ubicacion>congelador</ubicacion>
        <caducidad>2027-01-15</caducidad>
        <notas>Descongelar en la nevera antes de cocinar.</notas>
    </alimento>

    <!-- Productos de la despensa -->
    <alimento tipo="cereal" cantidad="500" unidad="g" estado="cerrado">
        <nombre>Pasta</nombre>
        <ubicacion>despensa</ubicacion>
        <caducidad tipo="preferente">2027-06-20</caducidad>
    </alimento>

    <alimento tipo="conserva" cantidad="2" unidad="ud">
        <nombre>Tomate triturado</nombre>
        <ubicacion>despensa</ubicacion>
        <caducidad tipo="preferente">2027-03-15</caducidad>
    </alimento>
</despensa>
```

Este ejemplo incorpora:

- Los tres tipos de fecha (caducidad, preferente y estimada)
- Atributos
- Elementos opcionales
- Diferentes tipos de alimentos
- Diferentes ubicaciones
- Fechas de apertura
- Estados
- Comentarios
- Notas

Una aplicación podría leer este documento y ordenar los alimentos según su fecha de caducidad.

He comprobado los ejemplos de este apartado y el del supuesto práctico (2.4) con el validador en línea XML Validation, y todos están bien formados. Este validador solo comprueba las reglas generales de XML, no las reglas de validación propias de ZeroWasteML del apartado 3.12.

5. Ventajas y limitaciones

5.1. Ventajas

Especialización

ZeroWasteML está diseñado específicamente para representar información sobre alimentos.

Fácil de leer

Su estructura permite que una persona pueda entender los datos directamente.

Fácil de procesar

Una aplicación puede leer las etiquetas y obtener la información de cada alimento.

Separación entre datos y lógica

El archivo almacena los datos mientras que una aplicación externa se encarga de procesarlos.

Reutilización

El mismo archivo podría ser utilizado por diferentes aplicaciones compatibles con ZeroWasteML.

5.2. Limitaciones

Es un lenguaje muy específico

ZeroWasteML está pensado para alimentos y no para cualquier tipo de información.

Necesita una aplicación externa

El lenguaje no calcula fechas, no envía avisos y no realiza acciones por sí mismo.

No es la mejor opción para listas muy sencillas

Si solo quiero apuntar cuatro cosas, una hoja de cálculo o una nota en el móvil es más rápido. Escribir etiquetas a mano es lento.

XML podría utilizarse para el mismo objetivo

Técnicamente, también sería posible almacenar esta información utilizando XML directamente.

JSON puede ser más práctico en algunos casos

Para aplicaciones web y APIs, JSON puede resultar más sencillo y habitual.

5.3. Resumen de utilidad

Uso	ZeroWasteML
Gestionar alimentos de una vivienda	Adecuado
Controlar fechas de caducidad mediante una aplicación	Adecuado
Compartir información sobre alimentos	Adecuado
Crear páginas web	No es su objetivo
Almacenar cualquier tipo de información	No es su objetivo
Crear una API moderna	JSON puede ser más práctico

6. Inspiración y referencias

Las principales referencias utilizadas para diseñar ZeroWasteML son las siguientes:

Cooklang
Cooklang – The recipe markup language.
Disponible en: https://cooklang.org/
Me sirvió de ejemplo de lenguaje de marcas pensado para un ámbito concreto (la cocina) y con una sintaxis sencilla.

W3C (World Wide Web Consortium)
Scalable Vector Graphics (SVG).
Disponible en: https://www.w3.org/Graphics/SVG/
Ejemplo de lenguaje basado en XML, igual que ZeroWasteML.

AESAN (Agencia Española de Seguridad Alimentaria y Nutrición)
¿Sabes la diferencia entre la fecha de consumo preferente y la fecha de caducidad?
Disponible en:
https://www.aesan.gob.es/AECOSAN/web/noticias_y_actualizaciones/noticias/2022/fechas.htm
Base para el atributo tipo del elemento <caducidad>.

BOE.
Ley 1/2025, de 1 de abril, de prevención de las pérdidas y el desperdicio alimentario.
Disponible en: https://www.boe.es/buscar/act.php?id=BOE-A-2025-6597

W3C (World Wide Web Consortium).
Extensible Markup Language (XML) 1.0.
Disponible en: https://www.w3.org/TR/xml/

ISO (International Organization for Standardization).
ISO 8601 — Date and time format.
Disponible en: https://www.iso.org/iso-8601-date-and-time-format.html

XML Validation
XML Validation – Validate XML files.
Disponible en: https://www.xmlvalidation.com/

Conclusión

ZeroWasteML nace de un problema cotidiano: los alimentos que se caducan en casa porque nadie se acuerda de ellos. Para resolverlo he diseñado un vocabulario basado en XML que guarda qué alimentos hay, dónde están, cuánto queda y, sobre todo, qué fecha tienen y de qué tipo es esa fecha.

Al ser un lenguaje basado en XML, cualquier programa que ya lea XML puede leer un archivo .zwml, y las reglas de validación garantizan que dos aplicaciones distintas entiendan los datos de la misma forma. Esto lo hace útil tanto en casa, con una aplicación que avise de lo que va a caducar, como entre negocios, por ejemplo, para enviar a un banco de alimentos los productos que todavía se pueden donar.